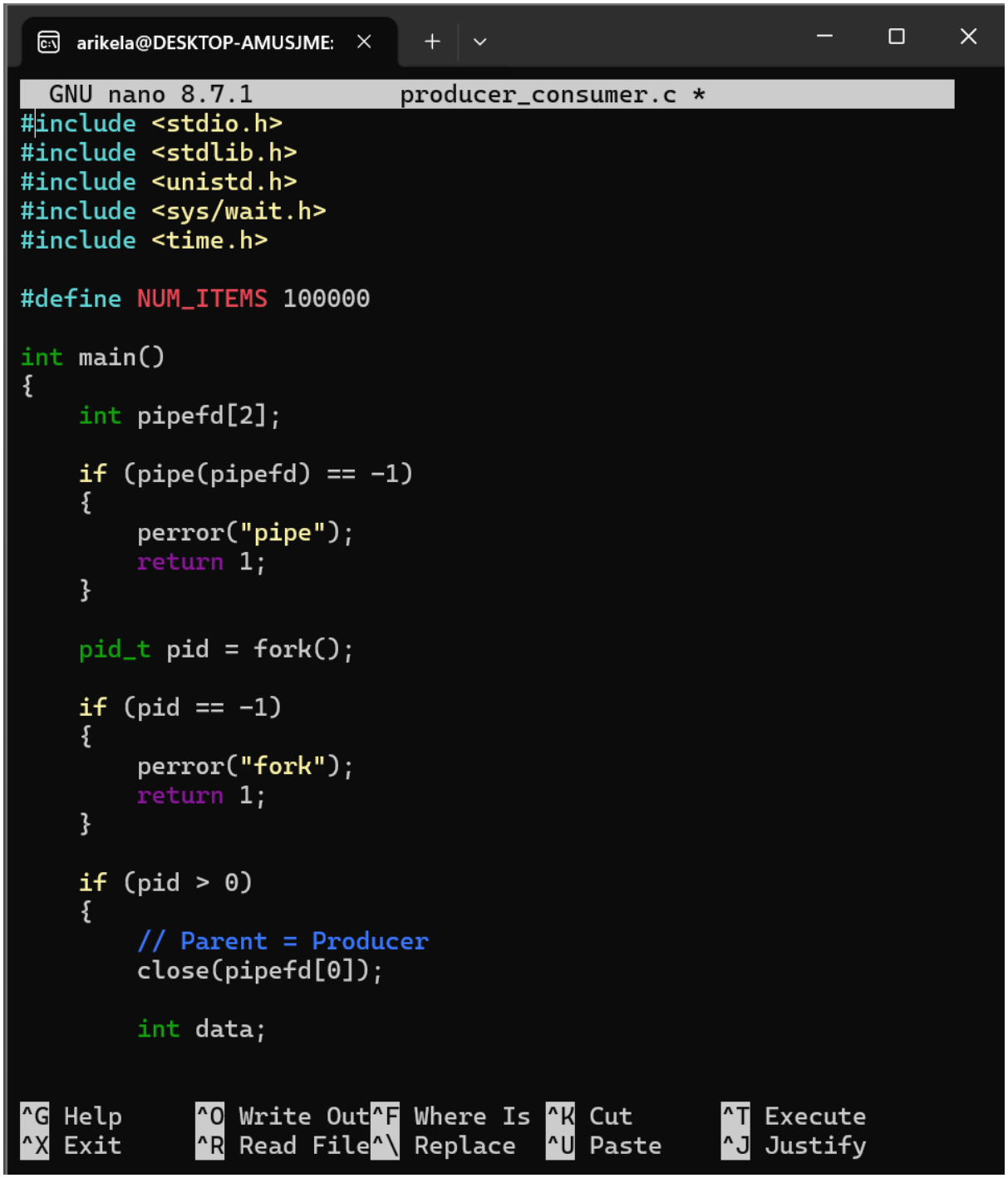
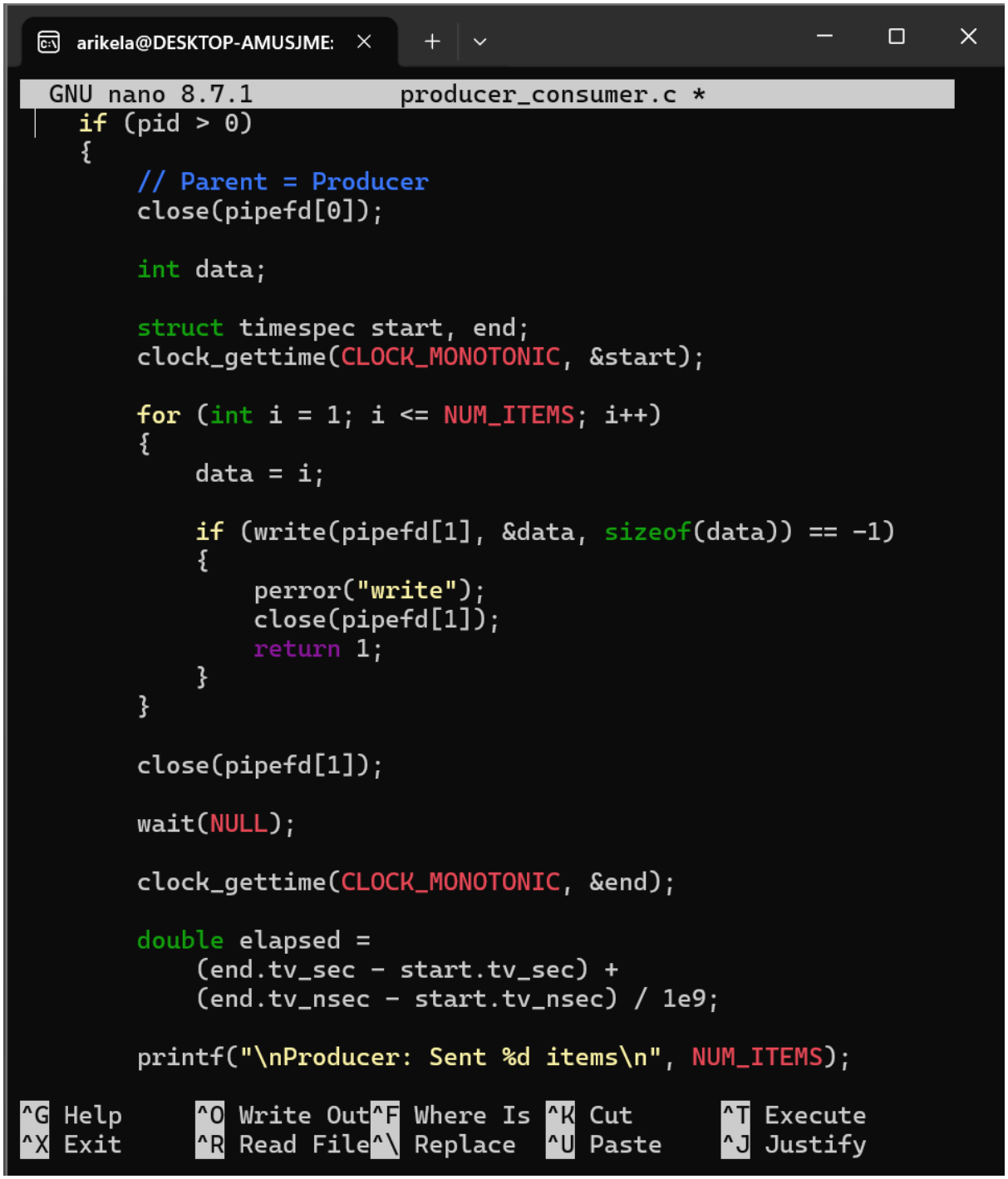
Practical Week-5

Task-1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.



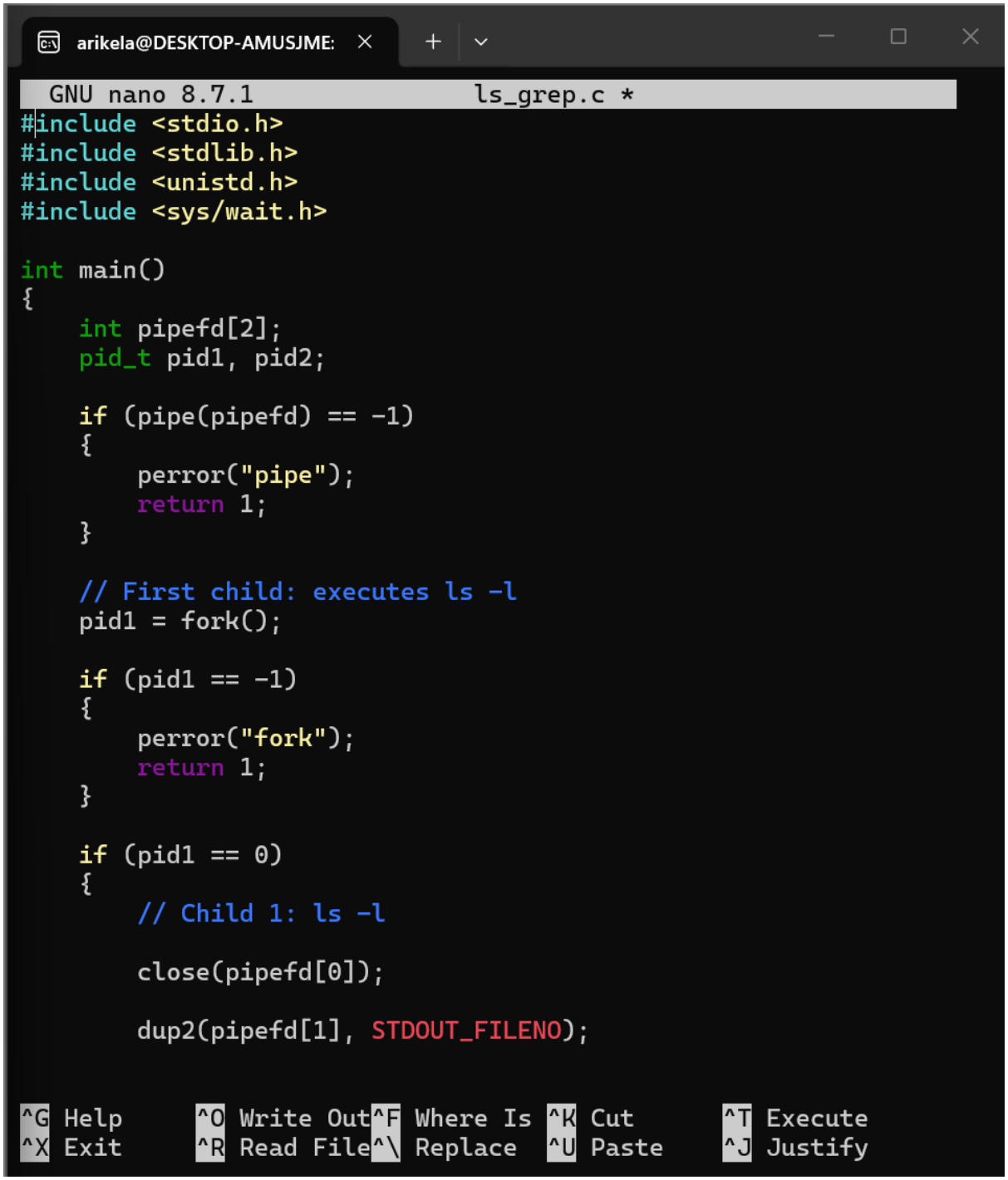

arikela@DESKTOP-AMUSJME:
GNU nano 8.7.1 producer_consumer.c *
if (pid > 0)
{
// Parent = Producer
close(pipefd[0]);
int data;
struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);
for (int i = 1; i <= NUM_ITEMS; i++)
{
data = i;
if (write(pipefd[1], &data, sizeof(data)) == -1)
{
perror("write");
close(pipefd[1]);
return 1;
}
}
close(pipefd[1]);
wait(NULL);
clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed =
(end.tv_sec - start.tv_sec) +
(end.tv_nsec - start.tv_nsec) / 1e9;
printf("\nProducer: Sent %d items\n", NUM_ITEMS);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

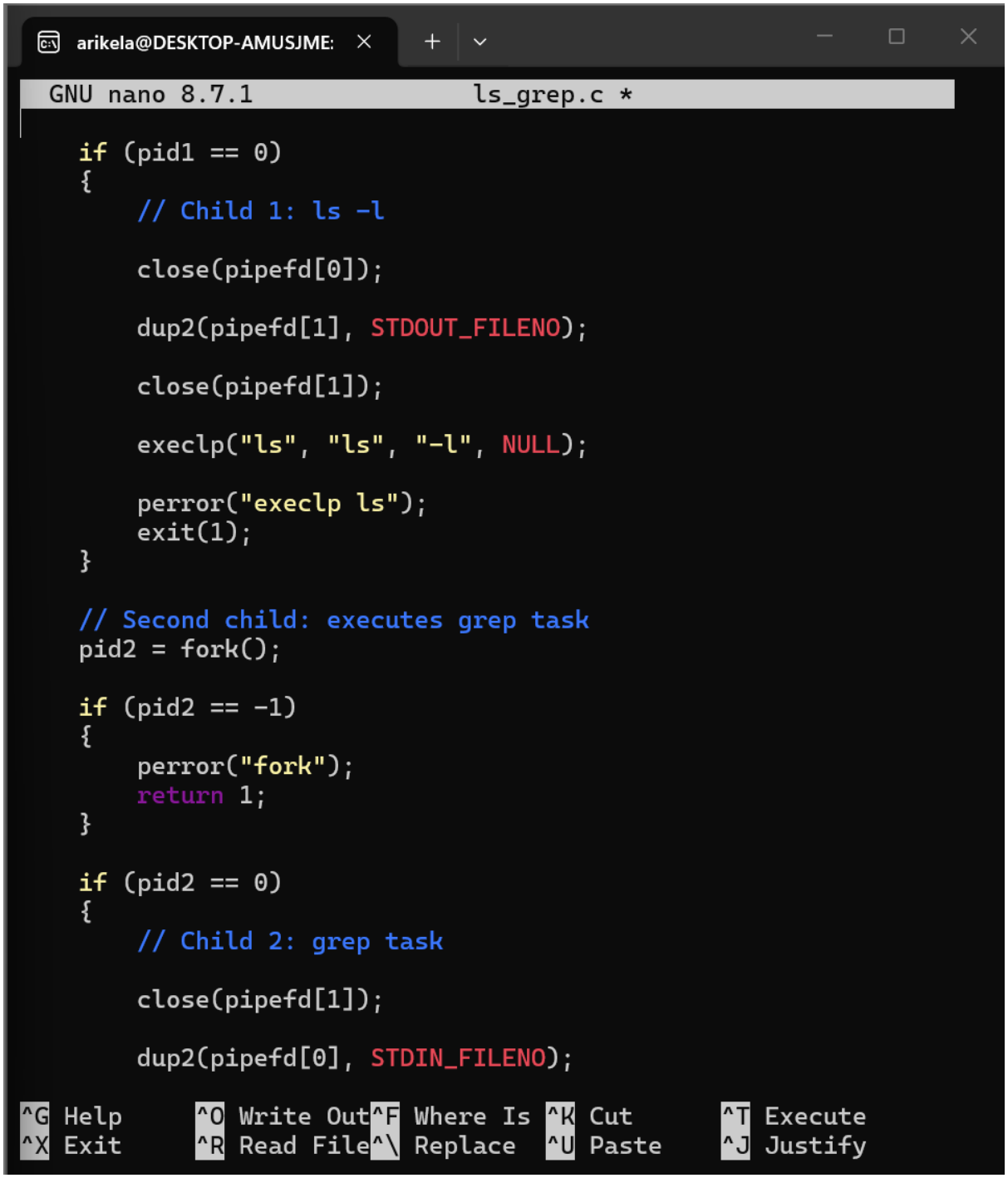

arikela@DESKTOP-AMUSJME:
GNU nano 8.7.1 producer_consumer.c *
double elapsed =
(end.tv_sec - start.tv_sec) +
(end.tv_nsec - start.tv_nsec) / 1e9;
printf("\nProducer: Sent %d items\n", NUM_ITEMS);
printf("Communication time: %.6f seconds\n", elapsed);
printf("Communication rate: %.2f items/second\n",
NUM_ITEMS / elapsed);
}
else
{
// Child = Consumer
close(pipefd[1]);
int data;
int count = 0;
long long sum = 0;
while (read(pipefd[0], &data, sizeof(data)) > 0)
{
sum += data;
count++;
}
close(pipefd[0]);
printf("Consumer: Received %d items\n", count);
printf("Consumer: Sum = %lld\n", sum);
}
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

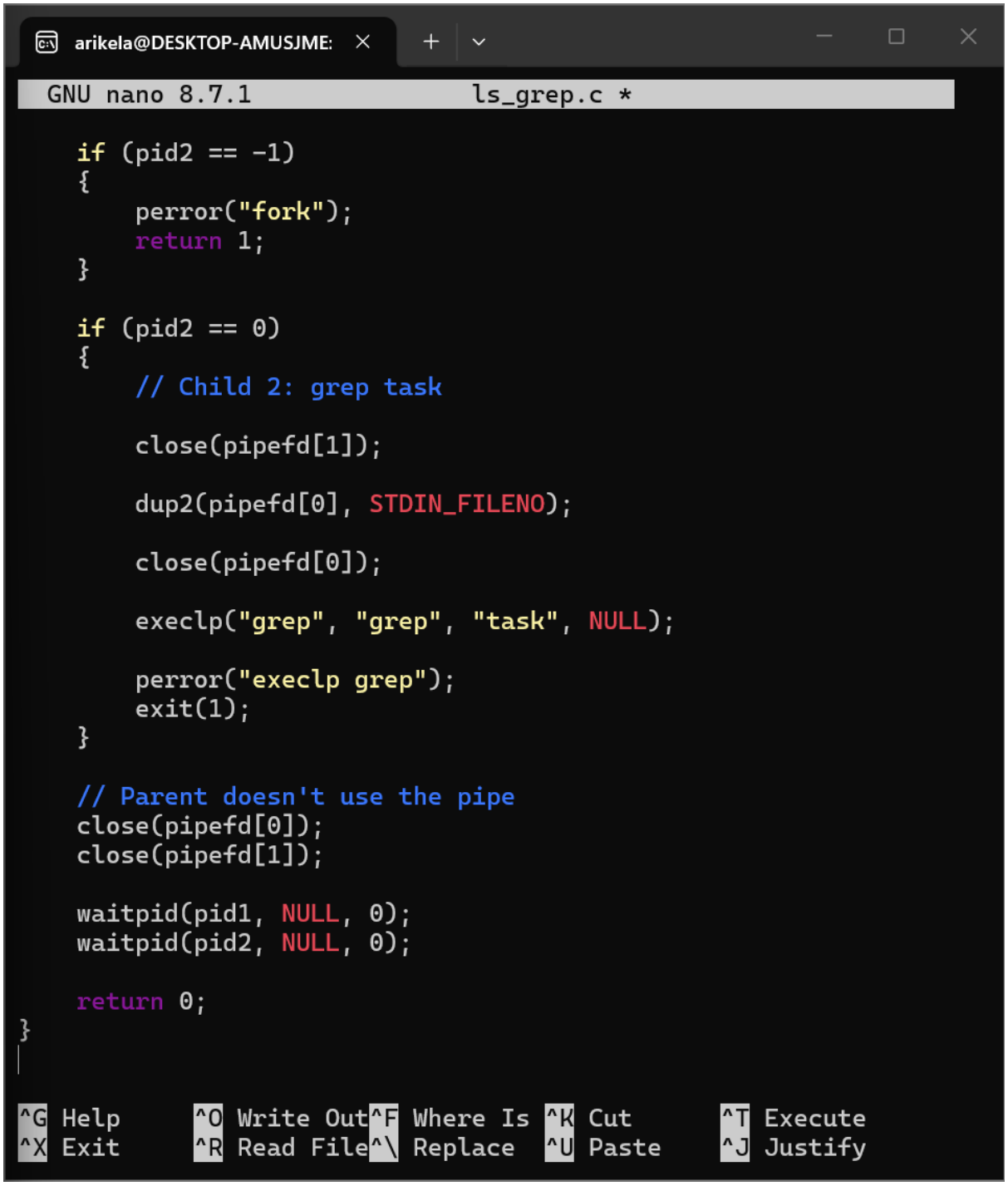
```
arikela@DESKTOP-AMUSJME:~$ mkdir week5prac
arikela@DESKTOP-AMUSJME:~$ cd week5prac
arikela@DESKTOP-AMUSJME:~/week5prac$ pwd
/home/arikela/week5prac
arikela@DESKTOP-AMUSJME:~/week5prac$ nano producer_consumer.c
arikela@DESKTOP-AMUSJME:~/week5prac$ gcc producer_consumer.c -o
producer_consumer
arikela@DESKTOP-AMUSJME:~/week5prac$ ls
producer_consumer  producer_consumer.c
arikela@DESKTOP-AMUSJME:~/week5prac$ ./producer_consumer
Consumer: Received 100000 items
Consumer: Sum = 5000050000

Producer: Sent 100000 items
Communication time: 0.033536 seconds
Communication rate: 2981898.86 items/second
arikela@DESKTOP-AMUSJME:~/week5prac$
```

Task-2:

Develop a program that executes the equivalent of the shell command: Is -l grep using fork(), pipe(), dup2(), and exec() system calls.


arikela@DESKTOP-AMUSJME:
GNU nano 8.7.1 ls_grep.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
int pipefd[2];
pid_t pid1, pid2;
if (pipe(pipefd) == -1)
{
perror("pipe");
return 1;
}
// First child: executes ls -l
pid1 = fork();
if (pid1 == -1)
{
perror("fork");
return 1;
}
if (pid1 == 0)
{
// Child 1: ls -l
close(pipefd[0]);
dup2(pipefd[1], STDOUT_FILENO);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


arikela@DESKTOP-AMUSJME:
GNU nano 8.7.1 ls_grep.c *
if (pid1 == 0)
{
// Child 1: ls -l
close(pipefd[0]);
dup2(pipefd[1], STDOUT_FILENO);
close(pipefd[1]);
execlp("ls", "ls", "-l", NULL);
perror("execlp ls");
exit(1);
}
// Second child: executes grep task
pid2 = fork();
if (pid2 == -1)
{
perror("fork");
return 1;
}
if (pid2 == 0)
{
// Child 2: grep task
close(pipefd[1]);
dup2(pipefd[0], STDIN_FILENO);
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


arikela@DESKTOP-AMUSJME:
GNU nano 8.7.1 ls_grep.c *
if (pid2 == -1)
{
perror("fork");
return 1;
}
if (pid2 == 0)
{
// Child 2: grep task
close(pipefd[1]);
dup2(pipefd[0], STDIN_FILENO);
close(pipefd[0]);
execlp("grep", "grep", "task", NULL);
perror("execlp grep");
exit(1);
}
// Parent doesn't use the pipe
close(pipefd[0]);
close(pipefd[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
return 0;
}
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
arikela@DESKTOP-AMUSJME:~/week5prac$ nano ls_grep.c
arikela@DESKTOP-AMUSJME:~/week5prac$ gcc ls_grep.c -o ls_grep
arikela@DESKTOP-AMUSJME:~/week5prac$ ls
ls_grep    producer_consumer    task_test.txt
ls_grep.c  producer_consumer.c
arikela@DESKTOP-AMUSJME:~/week5prac$ ls -l
total 40
-rwxr-xr-x 1 arikela arikela 16304 Sep 12 06:24 ls_grep
-rw-r--r-- 1 arikela arikela  1346 Sep 12 06:23 ls_grep.c
-rwxr-xr-x 1 arikela arikela 16360 Sep 12 05:31 producer_consume
r
-rw-r--r-- 1 arikela arikela  1645 Sep 12 05:30 producer_consume
r.c
-rw-r--r-- 1 arikela arikela     0 Sep 12 06:02 task_test.txt
arikela@DESKTOP-AMUSJME:~/week5prac$ ls -l | grep task
-rw-r--r-- 1 arikela arikela     0 Sep 12 06:02 task_test.txt
arikela@DESKTOP-AMUSJME:~/week5prac$ ./ls_grep
-rw-r--r-- 1 arikela arikela     0 Sep 12 06:02 task_test.txt
arikela@DESKTOP-AMUSJME:~/week5prac$
```